Universidad Nacional de Ingeniería

Área de Conocimiento de Tecnología de la Información y Comunicación (DACTIC)

Programa Académico de Ingeniería de Sistemas

Desarrollo de una plataforma digital de comercio electrónico y gestión logística para la microempresa Rapitrip.

Entregable final de proyecto para la asignatura de diseño de sistemas de internet.

Elaborado por:

Br. Richard Alexander Malespìn Tinoco	Br. Marlyng Elizabeth Miranda Navarrete	Br. Moisés Abraham Zelaya Maldonado
N° de Carnet: 2022-0323I	N° de Carnet: 2022-0315I	N° de Carnet: 2022-0225I

Tutor:

Cristopher Alexander Chávez Larios

11 de noviembre de 2026,

Managua, Nicaragua

Índice

Capítulo I: Análisis de requerimientos y Diagramas UWE/UML 4

Introducción 1

Objetivos 1

Objetivo general 1

Objetivos específicos 1

1.1. Requerimientos Funcionales y No funcionales 2

1.1.1. Requerimientos funcionales 2

1.1.2. Requerimientos no funcionales 19

1.2. Diagramas de Casos de Uso 26

1.2.1. Diagrama de caso de usos del negocio 26

1.2.2. Diagrama de caso de usos del sistema 27

1.3. Diagrama de Clases 28

1.4. Diagramas de navegación y presentación (Prototipo de figma) 3

1.4.1. Archivo de figma vista general de los protitipos y capas del sistema 3

1.4.2. Diagrama de navegación y presentación del cliente en Web 3

1.4.3. Diagrama de navegación y presentación de la consola de operación en Web .3

1.4.4. Diagrama de navegación y presentación del repartidor en Movil 3

1.5. Diagramas de proceso 5

1.5.1. Diagrama de proceso | Realizar pedido 5

1.5.2. Diagrama de proceso | Registrar producto 6

1.6. Modelo Entidad – Relación 7

Capítulo II: Arquitectura de Software 9

2.1. Enfoque general 10

2.2. Aplicaciones cliente y código compartido 11

2.3. Backend y organización de los datos 11

2.4. Recorrido de una operación 12

2.5. Seguridad y control de acceso 13

2.6. Ventajas y limitaciones del diseño 13

2.7. Evolución prevista 14
Capítulo III: Desarrollo de backend 15
3.1. Tecnología utilizada 16
3.1.1. Tecnologías por capa 16
3.1.2. Supabase como backend central 17
3.2. Patrón de diseño que se utilizó 18
3.2.1. Backend modular centrado en la base de datos 18
3.2.2. Patrones aplicados 19
3.3. Estrategias de desarrollo 20
3.3.1. Metodología 20
3.3.2. Secuencia de construcción 20
3.3.3. Desarrollo guiado por pruebas 20
3.3.4. Seguridad desde el diseño 21
3.3.5. Control de cambios y entornos 21
3.3.6. Fases del proyecto 22
5. URL de proyecto en github 22

Capítulo I: Análisis de requerimientos y Diagramas UWE/UML

Introducción

Rapitrip desarrolla una plataforma de comercio electrónico y gestión logística que relaciona a clientes, comercios afiliados, personal operativo y repartidores. Sus procesos abarcan la publicación de productos, el control de inventario, la compra, el pago y la entrega. Por ello, el sistema requiere información compartida y permisos diferenciados para coordinar cada pedido desde su creación hasta su cierre.

Este entregable final integra los tres avances anteriores y actualiza sus decisiones conforme al desarrollo alcanzado. Reúne los requerimientos, los casos de uso y los modelos de procesos, clases, navegación y datos; además, explica la arquitectura de tres aplicaciones web sobre un backend compartido en Supabase. Así, el análisis y el diseño se relacionan con los componentes construidos y con las estrategias técnicas y de seguridad que sustentan su funcionamiento.

Objetivos

Objetivo general

Diseñar y desarrollar una plataforma digital para Rapitrip que integre el comercio electrónico y la gestión logística, permita la participación de clientes, comercios, personal operativo y repartidores, y mantenga seguros y consistentes el catálogo, el inventario, los pedidos, los pagos y las entregas.

Objetivos específicos

1. Definir requerimientos funcionales y no funcionales verificables a partir de los procesos y actores de Rapitrip.
2. Modelar casos de uso, procesos, clases, navegación y datos para mantener la trazabilidad del diseño.
3. Describir la arquitectura de tres aplicaciones web y un backend compartido en Supabase, con responsabilidades delimitadas.
4. Integrar catálogo, inventario, pedidos, pagos y entregas mediante flujos de información consistentes.
5. Aplicar controles de acceso y trazabilidad, y contrastar los modelos con el desarrollo y las pruebas.

1.1. Requerimientos Funcionales y No funcionales

1.1.1. Requerimientos funcionales

Cada módulo agrupa los requerimientos funcionales según el proceso de negocio que atienden, siguiendo la misma organización de la propuesta tecnológica de Rapitrip. La columna Origen indica si el requerimiento proviene de uno ya existente en el primer entregable (con su código original) o si es un requerimiento nuevo exigido por la plataforma multi-rol. La columna Fase ubica el requerimiento dentro del plan de implementación por fases (F1 a F7); “B y C” señala que solo se construye si Rapitrip desarrolla la aplicación del repartidor. La columna Prior. es A (alta) o M (media).

Módulo IDA: Identidad y acceso

Aplicación o área responsable: consola web, tienda web y aplicación web del repartidor; control de acceso transversal.

Código	Requerimiento	Datos	Criterio de aceptación
RF-IDA-01	Registrar la cuenta del cliente y verificar su correo o teléfono.	Actor: Cliente	Una cuenta no verificada no puede hacer pedidos.
RF-IDA-02	Iniciar y cerrar sesión con acceso limitado a las funciones del rol.	Actor: Todos	Un usuario sin rol activo ve “Cuenta sin acceso”; ningún rol accede a funciones de otro.
RF-IDA-03	Dar de alta personal por invitación de un solo uso.	Actor: Administrador, Superadministrador	La persona define su propia contraseña; el administrador nunca la conoce; solo un superadministrador crea administradores.

Código	Requerimiento	Datos	Criterio de aceptación
RF-IDA-04	Asignar, suspender, reactivar y revocar roles, con alcance por comercio.	Actor: Administrador	Nadie asigna un rol igual o superior al suyo; suspender el rol de comercio no afecta el acceso como cliente. Una persona puede tener los roles de coordinador de pedidos y de cajero a la vez.
RF-IDA-05	Consultar y modificar los datos de la cuenta.	Actor: Todos los roles	Cada usuario edita solo sus datos y solo los campos habilitados para su rol.

Módulo SUP: Supervisión de la operación

Aplicación o área responsable: consola web, áreas /admin y /operaciones.

Código	Requerimiento	Datos	Criterio de aceptación
RF-SUP-01	Consultar la auditoría de acciones y el registro de inicios de sesión.	Actor: Administrador	Cada inicio de sesión guarda cuenta, fecha, hora y dispositivo; la auditoría se filtra por actor, acción y objetivo.

Código	Requerimiento	Datos	Criterio de aceptación
RF-SUP-02	Configurar los parámetros operativos: modo de asignación, tiempos límite (confirmación del comercio, preparación, respuesta a ofertas) y límite de cancelación del cliente.	Actor: Administrador	Un cambio de parámetro se aplica sin modificar código y queda auditado; el tiempo de confirmación del comercio es de 15 minutos por defecto.
RF-SUP-03	Eliminar o anonimizar datos personales de cuentas inactivas y atender solicitudes de los titulares.	Actor: Administrador	Cada eliminación queda registrada con fecha y responsable, sin conservar datos personales identificables.
RF-SUP-04	Supervisar la operación: pedidos por estado y tienda, tiempos, cancelaciones, tasa de faltantes y comercios sin actualizar catálogo.	Actor: Administrador	El administrador puede suspender un comercio reincidente en faltantes. En F3 se muestran pedidos por estado y tienda y comercios sin actualizar catálogo; en F6 se suman los tiempos por etapa y los indicadores completos.

Código	Requerimiento	Datos	Criterio de aceptación
RF-SUP-05	Procesar pedidos desde un tablero en vivo con filtros por tienda, zona y antigüedad, y alertas por tiempo excedido.	Actor: Coordinador de pedidos	El tablero refleja un cambio de estado sin recargar la página. Antes de F6 el coordinador de pedidos trabaja con una lista de pedidos en la consola.
RF-SUP-06	Gestionar la cola de excepciones: tomar, resolver y comentar. Nueve tipos: faltante reportado, confirmación vencida, preparación excedida, sin repartidor disponible, oferta sin respuesta, entrega retrasada, solicitud de cancelación del cliente, entrega fallida y manual.	Actor: Coordinador de pedidos	Cada excepción conserva causa, responsable y resolución. Los tipos “oferta sin respuesta” y “sin repartidor disponible” por oferta automática solo existen si se desarrolla la aplicación del repartidor.

Código	Requerimiento	Datos	Criterio de aceptación
RF-SUP-07	Resolver faltantes y confirmaciones vencidas: quitar el ítem y reembolsar, sustituir con confirmación del cliente, confirmar en nombre del comercio tras contactarlo, o cancelar el pedido.	Actor: Coordinador de pedidos	Una sustitución no se aplica hasta que el cliente la acepta; toda confirmación en nombre del comercio queda auditada; al resolver, el pedido continúa o se cancela. Cuando la resolución exige un reembolso, el coordinador lo solicita y el cajero lo registra (RF-COB-02).
RF-SUP-08	Registrar notas internas y consultar la línea de tiempo del pedido.	Actor: Coordinador de pedidos	La línea de tiempo combina estados y, desde F5, eventos de entrega en orden cronológico.

Módulo COM: Comercios y catálogo

Aplicación o área responsable: consola web, áreas /admin y /comercio.

Código	Requerimiento	Datos	Criterio de aceptación
RF-COM-01	Solicitar afiliación como comercio con datos del negocio y documentos.	Actor: Cliente registrado	La solicitud queda “pendiente” con sus documentos en almacenamiento privado.
RF-COM-02	Aprobar o rechazar solicitudes de comercio con motivo.	Actor: Administrador	Al aprobar (con convenio firmado) se crean el comercio y su rol; al rechazar se guarda el motivo.
RF-COM-03	Administrar comercios y tiendas: registro, consulta, actualización y suspensión.	Actor: Administrador	Un comercio suspendido no recibe pedidos nuevos.
RF-COM-04	Administrar las tiendas propias: horario y apertura o cierre temporal.	Actor: Comercio	Una tienda cerrada no aparece disponible para pedidos.

Código	Requerimiento	Datos	Criterio de aceptación
RF-COM-05	Administrar el catálogo de la tienda: productos, variantes, precio, SKU, imágenes y publicación.	Actor: Comercio	Un comercio solo ve y edita el catálogo de sus tiendas.
RF-COM-06	Mantener el catálogo maestro: categorías, marcas, tipos de producto y atributos.	Actor: Administrador	Los comercios usan el catálogo maestro; solo el administrador lo modifica.
RF-COM-07	Consultar las métricas propias del comercio: ventas, tiempo de preparación y tasa de faltantes.	Actor: Comercio	Un comercio solo ve sus propias métricas.

Módulo INV: Inventario

Aplicación o área responsable: consola web, área /comercio; lógica en el servidor.

Código	Requerimiento	Datos	Criterio de aceptación
RF-INV-01	Gestionar inventario y disponibilidad: ajuste de stock, movimientos, alerta de stock bajo y ocultar agotados.	Actor: Comercio	Al llegar a 0 el producto se oculta del catálogo; toda variación deja un movimiento registrado.
RF-INV-02	Descontar el stock al generar el pedido y devolverlo si se cancela.	Actor: Sistema	Con stock insuficiente no se crea el pedido; al cancelar, el stock vuelve al inventario.
RF-INV-03	Reportar faltantes sobre uno o más ítems de un pedido, en lugar de confirmarlo.	Actor: Comercio	Al reportar se abre una excepción de tipo “faltante reportado”, se registra un ajuste de inventario, el pedido sigue “En confirmación” hasta resolverse, y el reporte cuenta para la tasa de faltantes del comercio.

Módulo PED: Carrito y pedidos

Aplicación o área responsable: tienda web (origen del pedido) y funciones del backend. La lógica del pedido se incorporó desde F3 y la interfaz de compra se desarrolló en F4.

Código	Requerimiento	Datos	Criterio de aceptación
RF-PED-01	Calcular el costo de envío según la zona del tabulador.	Actor: Sistema	El envío coincide con la zona (1 a 5) de la ruta calculada y con el tabulador vigente.
RF-PED-02	Generar un pedido por cada comercio del carrito, validando stock y datos requeridos.	Actor: Cliente	El pedido conserva detalle, precio y dirección del momento de la compra.
RF-PED-03	Registrar pedidos recibidos por canales externos.	Actor: Coordinador de pedidos	El pedido manual recibe identificador, estado y trazabilidad como cualquier otro.

Código	Requerimiento	Datos	Criterio de aceptación
RF-PED-04	Controlar los estados del pedido con historial de cada cambio: En confirmación, Confirmado, En preparación, Listo, En entrega, Entregado y Cancelado.	Actor: Sistema	Solo se permiten las transiciones definidas (véase el apartado 5); cada cambio guarda actor, fecha y motivo.
RF-PED-05	Aceptar automáticamente el pedido: al generarse con pago válido (contra entrega o pago en línea aprobado) queda “En confirmación” sin intervención del personal.	Actor: Sistema	Con pago en línea fallido o vencido el pedido pasa a “Cancelado”, devuelve el stock y nunca llega al comercio.

Código	Requerimiento	Datos	Criterio de aceptación
RF-PED-06	Confirmar el pedido: el comercio confirma la disponibilidad dentro de 15 minutos y el pedido pasa a “Confirmado”.	Actor: Comercio	El plazo corre desde que el pedido es visible para el comercio; sin respuesta a los 15 minutos se abre la excepción “confirmación vencida” y el pedido sigue “En confirmación”.
RF-PED-07	Actualizar la preparación del pedido: iniciar la preparación y dejarlo “Listo”.	Actor: Comercio	Solo el comercio del pedido puede iniciar la preparación (desde “Confirmado”) y marcarlo “Listo” (desde “En preparación”); se registra el tiempo de preparación.
RF-PED-08	Cancelar pedidos: el cliente dentro del límite configurado; el coordinador de pedidos y el administrador en cualquier estado no final.	Actor: Cliente, Coordinador de pedidos, Administrador	La cancelación devuelve el stock y, si corresponde un reembolso, el cajero lo registra (RF-COB-02). El cliente la solicita desde la tienda web (F4).

Módulo CLI: Clientes y direcciones

Aplicación o área responsable: tienda web del cliente (Etapa 1).

Código	Requerimiento	Datos	Criterio de aceptación
RF-CLI-01	Consultar la oferta comercial: catálogo público, búsqueda y detalle, solo con productos disponibles.	Actor: Cliente	La búsqueda devuelve coincidencias o informa que no hay resultados; funciona sin sesión y el catálogo público es indexable.
RF-CLI-02	Gestionar direcciones de entrega.	Actor: Cliente Origen: RF17 (ajuste) Fase: F4 Prioridad: A	El cliente crea, edita y elimina sus direcciones, con mapa, y no accede a las de otros.
RF-CLI-03	Gestionar el carrito: agregar, modificar cantidad, eliminar y ver resumen.	Actor: Cliente Origen: RF5 (ajuste) Fase: F4 Prioridad: A	El carrito agrupa productos por comercio.
RF-CLI-04	Enviar mensajes de ayuda y comentarios de opinión, con seguimiento de su atención.	Actor: Cliente Origen: RF19 Fase: F4 Prioridad: M	El mensaje queda registrado con su estado de atención.

Código	Requerimiento	Datos	Criterio de aceptación
RF-CLI-05	Reportar inconsistencias entre el catálogo y lo recibido.	Actor: Cliente Origen: Nuevo Fase: F4 Prioridad: M	El reporte llega al coordinador de pedidos y queda vinculado al pedido.

Módulo SEG: Seguimiento del pedido

Aplicación o área responsable: tienda web del cliente (Etapa 1).

Código	Requerimiento	Datos	Criterio de aceptación
RF-SEG-01	Consultar pedidos y seguir su etapa en tiempo real. Con la aplicación del repartidor, el seguimiento incluye además el código de entrega y la ubicación del repartidor.	Actor: Cliente	La etapa del pedido se actualiza sin recargar. El código de entrega solo lo ve el cliente dueño del pedido.

Código	Requerimiento	Datos	Criterio de aceptación
RF-SEG-02	Confirmar o rechazar las sustituciones propuestas.	Actor: Cliente Origen: Nuevo Fase: F4 Prioridad: A	Cada sustitución queda aceptada o rechazada con fecha.

Módulo ENT: Asignación y entrega

Aplicación o área responsable: consola web, área de coordinación, y aplicación web del repartidor. La coordinación asigna y supervisa las entregas, mientras que el repartidor registra sus acciones y evidencias en la aplicación.

Código	Requerimiento	Datos	Criterio de aceptación
RF-ENT-01	Gestionar repartidores: alta por invitación, consulta, actualización, activación y desactivación.	Actor: Administrador	Un repartidor desactivado no recibe asignaciones ni, si existe la aplicación, ofertas ni inicia sesión.
RF-ENT-02	Activar y desactivar el turno; capturar la ubicación solo con el turno activo.	Actor: Repartidor	Sin turno activo no se registra ubicación; la ubicación solo la ven el pedido activo, el coordinador de pedidos y el administrador.

Código	Requerimiento	Datos	Criterio de aceptación
RF-ENT-03	Asignar entregas con un candidato sugerido y permitir la reasignación manual.	Actor: Coordinador de pedidos	En F3 el coordinador anota quién entrega. En F6 el sistema sugiere un candidato cuando el pedido pasa a “Confirmado” (no antes): sin la aplicación, el repartidor disponible que el coordinador marcó con menos pedidos activos; con la aplicación, el más cercano con turno activo. Toda reasignación guarda el motivo.
RF-ENT-04	Ofertar entregas de forma automática con cuenta regresiva y paso al siguiente candidato.	Actor: Sistema	Si nadie acepta, se abre la excepción “sin repartidor disponible”.
RF-ENT-05	Gestionar entregas: creación, estado y eventos asociados al pedido.	Actor: Sistema	Cada pedido tiene una sola entrega con estados: recogido, en ruta y entregado. Sin la aplicación, el coordinador de pedidos cambia esos estados a mano.

Código	Requerimiento	Datos	Criterio de aceptación
RF-ENT-06	Ejecutar entregas: ver asignadas, aceptar o rechazar ofertas, recoger y avanzar el estado.	Actor: Repartidor	Solo el repartidor asignado cambia el estado de su entrega.
RF-ENT-07	Consultar el destino y la ruta de la entrega.	Actor: Repartidor	La ruta se abre desde la entrega activa.
RF-ENT-08	Confirmar la entrega con el código de cuatro dígitos del cliente y una foto obligatoria.	Actor: Repartidor	Sin código válido y sin foto, la entrega no pasa a “entregado”. Sin la aplicación, el coordinador de pedidos confirma la entrega a mano y la plataforma no tiene código ni foto de entrega.
RF-ENT-09	Registrar y atender incidencias de entrega y de pedido.	Actor: Repartidor, Comercio, Coordinador de pedidos	Cada incidencia abre una excepción con tipo, autor y resolución.

Módulo COB: Cobros y liquidación

Aplicación o área responsable: motor de pedidos en el servidor y consola web, áreas /admin y /caja. La plataforma calcula, desglosa y registra; el pago a comercios y repartidores se hace fuera de ella, en persona o por transferencia. El cajero trabaja en el área /caja y el administrador conserva el cierre quincenal.

Código	Requerimiento	Datos	Criterio de aceptación
RF-COB-01	Registrar pagos y comprobantes: contra entrega y Stripe, con prepago obligatorio a más de 10 km.	Actor: Cliente, Sistema, Cajero	El pedido de zonas 4 y 5 solo avanza con el pago en línea confirmado. El cajero registra el cobro contra entrega y su comprobante. La tienda inicia una sesión de pago con Stripe y confirma el resultado mediante el webhook.
RF-COB-02	Registrar reembolsos y devoluciones.	Actor: Cajero, Administrador	Cada reembolso queda asociado al pago, con monto y motivo. El coordinador de pedidos lo solicita desde el pedido y el cajero lo registra.
RF-COB-03	Calcular la liquidación de cada comercio (pedidos entregados menos comisión y reversiones) con plazo máximo de 15 días, y registrar el pago como realizado.	Actor: Cajero, Administrador	El estado de cuenta del comercio suma los pedidos entregados menos comisión y reversiones, e indica la fecha límite de pago. La plataforma no ejecuta el pago: el cajero registra su fecha, medio y comprobante.

Código	Requerimiento	Datos	Criterio de aceptación
RF-COB-04	Calcular la cuenta de cada repartidor según el tabulador (entregas, monto a pagar y efectivo por rendir) y registrar el pago como realizado.	Actor: Cajero, Administrador	El monto por entrega coincide con la zona, y el saldo es lo ganado menos el efectivo no rendido. La plataforma no ejecuta el pago: el cajero lo registra.
RF-COB-05	Realizar el cierre quincenal entre los socios.	Actor: Administrador	El cierre lista ingresos, costos y reparto de la quincena.
RF-COB-06	Conciliar pedido, comprobante de pago y depósito bancario.	Actor: Cajero, Administrador	La plataforma cruza el pedido con el comprobante de Stripe, el cajero registra el depósito bancario, y las diferencias entre los tres registros se listan para que las revise el administrador.

1.1.2. Requerimientos no funcionales

Los requerimientos no funcionales son transversales: condicionan el comportamiento de la plataforma completa y no pertenecen a un módulo ni a una aplicación específica. Cada uno incluye un criterio de aceptación verificable por prueba, que sustituye las metas sin medición del primer entregable.

Código	Requerimiento no funcional	Origen y fase	Criterio de aceptación verificable
RNF-01	Tiempo de respuesta: las consultas principales (catálogo, estado del pedido, tablero) responden en menos de 3 segundos bajo la carga de referencia.	Origen: RNF1 Fase: F1-F6	Prueba de carga con la carga de referencia aprobada: 50 pedidos al mes con hasta 10 pedidos activos a la vez; el 95 % de las respuestas cumple el límite.
RNF-02	Disponibilidad mensual mínima del 99 %.	Origen: RNF2 Fase: F4, F7	Monitoreo de Supabase y de las aplicaciones web desplegadas en Vercel; el 99 % de un mes de 30 días permite hasta 7.2 horas de indisponibilidad.
RNF-03	Autorización por rol y alcance: ninguna función privilegiada responde a un usuario anónimo ni a un rol sin alcance.	Origen: RNF3 Fase: F1	La matriz de permisos (cada función por cada rol, incluido anónimo) pasa al 100 % en pruebas de base de datos. La matriz distingue al coordinador de pedidos y al cajero: el primero no registra cobros ni pagos y el segundo no cambia estados de pedidos.

Código	Requerimiento no funcional	Origen y fase	Criterio de aceptación verificable
RNF-04	Autenticación y sesión: verificación en dos pasos para administrador, superadministrador, coordinador de pedidos y cajero; cierre de sesión de la consola a los 15 minutos de inactividad para el personal de Rapitrip; contraseña propia de al menos ocho caracteres.	Origen: RNF3 Fase: F1, F7	El cierre por inactividad y la contraseña rigen desde F1. Una cuenta de personal sin segundo factor no accede a la consola en producción completa (F7, con la verificación en dos pasos de Supabase Auth).
RNF-05	Protección de datos personales: el código de entrega solo lo ve el cliente; la ubicación solo el pedido activo, el coordinador de pedidos y el administrador; documentos en almacenamiento privado; exportación masiva reservada, cifrada y registrada; cumplimiento de la Ley 787.	Origen: Nuevo Fase: F1	Pruebas de acceso por rol sobre cada dato sensible; toda exportación masiva queda registrada.

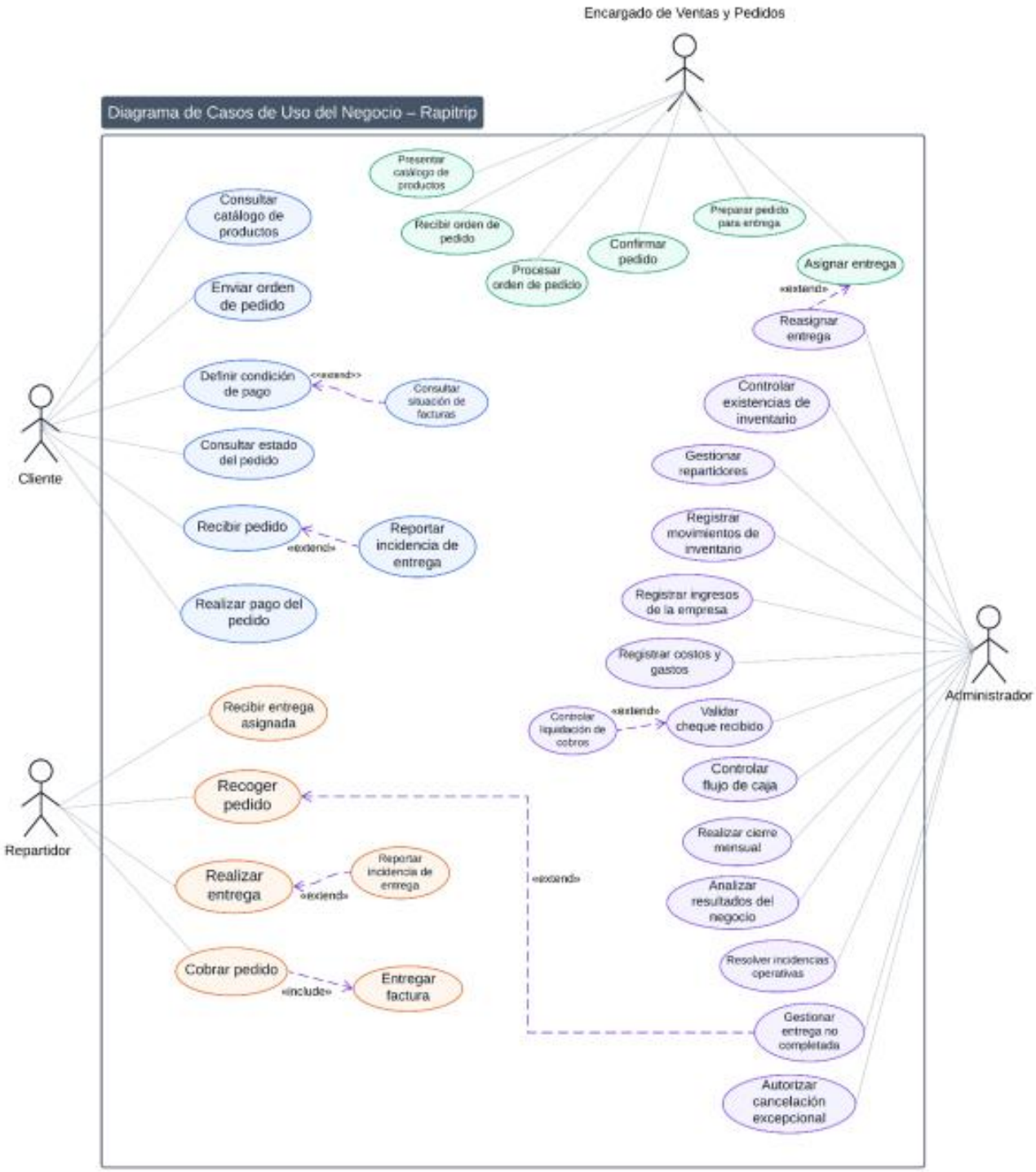
Código	Requerimiento no funcional	Origen y fase	Criterio de aceptación verificable
RNF-06	Auditoría y trazabilidad: cada cambio de estado, cobro, devolución y ajuste queda registrado con usuario y fecha, y ningún rol puede modificar el registro.	Origen: Nuevo Fase: F1	Un intento de modificar o borrar un registro de auditoría es rechazado en todos los roles.
RNF-07	Integridad y consistencia: pedido, descuento de stock y pago se registran en una sola transacción; el estado del pedido solo cambia por transiciones válidas.	Origen: RNF7 Fase: F3	Pruebas de transiciones válidas e inválidas; no existen pedidos sin detalle, entregas sin pedido ni stock negativo.
RNF-08	Respaldo y recuperación: respaldo diario con retención de 7 días y prueba de restauración trimestral en un entorno separado.	Origen: RNF8 Fase: F1, F4	Registro de cada prueba trimestral con fecha y resultado. Pérdida máxima aceptable: un día. Tiempo de recuperación: 24 horas.

Código	Requerimiento no funcional	Origen y fase	Criterio de aceptación verificable
RNF-09	Modularidad y mantenibilidad: backend compartido en Supabase, tres aplicaciones web (tienda, consola y repartidor), paquetes comunes, migraciones versionadas como fuente de verdad y documentación de despliegue y operación.	Origen: RNF4 Fase: F1	Todo cambio de esquema pasa por una migración versionada; el repositorio incluye documentación de despliegue.
RNF-10	Usabilidad: interfaces consistentes entre roles, con validaciones junto al campo y resultados vacíos explicados.	Origen: RNF5 Fase: F2-F6	Prueba con usuarios por rol; el 90 % de las tareas críticas se completa sin ayuda.
RNF-11	Compatibilidad: tienda, consola y aplicación del repartidor disponibles en navegadores modernos, con interfaces adaptadas a teléfono y computador según las tareas de cada actor.	Origen: RNF6 Fase: F4, F5	Pruebas de las tres aplicaciones web en navegadores y dispositivos representativos, incluidas las funciones de cámara y ubicación que utiliza la aplicación del repartidor.
RNF-12	Resiliencia ante conectividad intermitente: las operaciones del repartidor se reintentan sin duplicar registros.	Origen: RNF9 Fase: F5 (B y C)	Una foto o cambio de estado enviado sin conexión se registra una sola vez al recuperar la conexión.

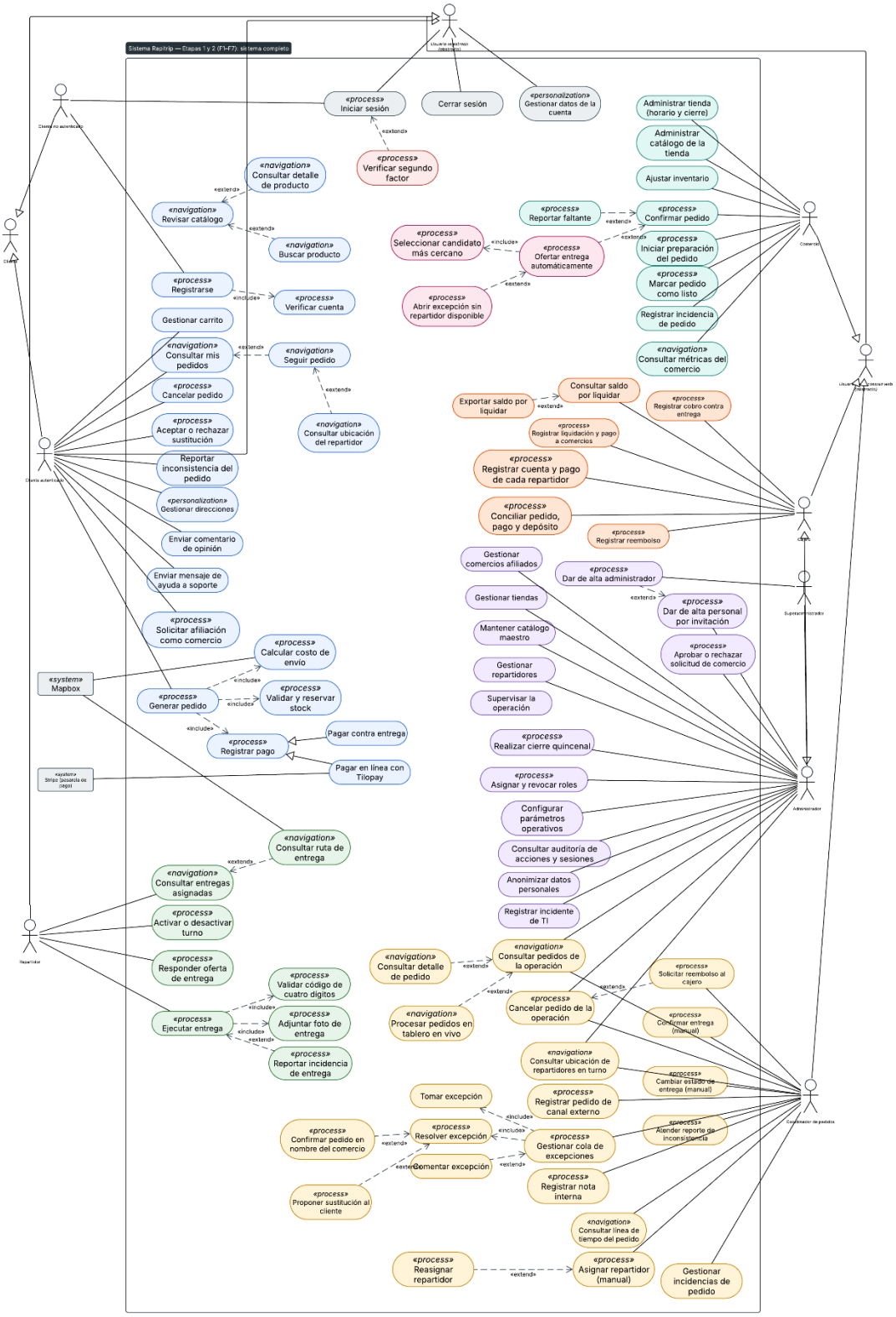
Código	Requerimiento no funcional	Origen y fase	Criterio de aceptación verificable
RNF-13	Actualización en tiempo real: tablero y seguimiento reflejan los cambios de estado sin recargar.	Origen: Nuevo Fase: F4, F6	Un cambio de estado aparece en el tablero y en el seguimiento del cliente en un máximo de 5 segundos.
RNF-14	Configurabilidad y escalabilidad: tiempos límite, modo de asignación, límite de cancelación y tabulador se ajustan sin cambiar código; comercios, zonas y repartidores se incorporan por configuración.	Origen: Nuevo Fase: F1, F6	Cambiar un parámetro o añadir un comercio no requiere una nueva versión de la aplicación.
RNF-15	Exactitud financiera: el desglose de cada pedido (comercio, repartidor, Rapitrip y comisión de la pasarela) suma exactamente lo pagado por el cliente.	Origen: Nuevo Fase: F3, F7	Para el pedido de ejemplo (producto C$450, zona 1): 405 + 40 + 55 = 500; y con Tilopay: 400.50 + 40 + 54.50 + 5.00 = 500.

1.2. Diagramas de Casos de Uso

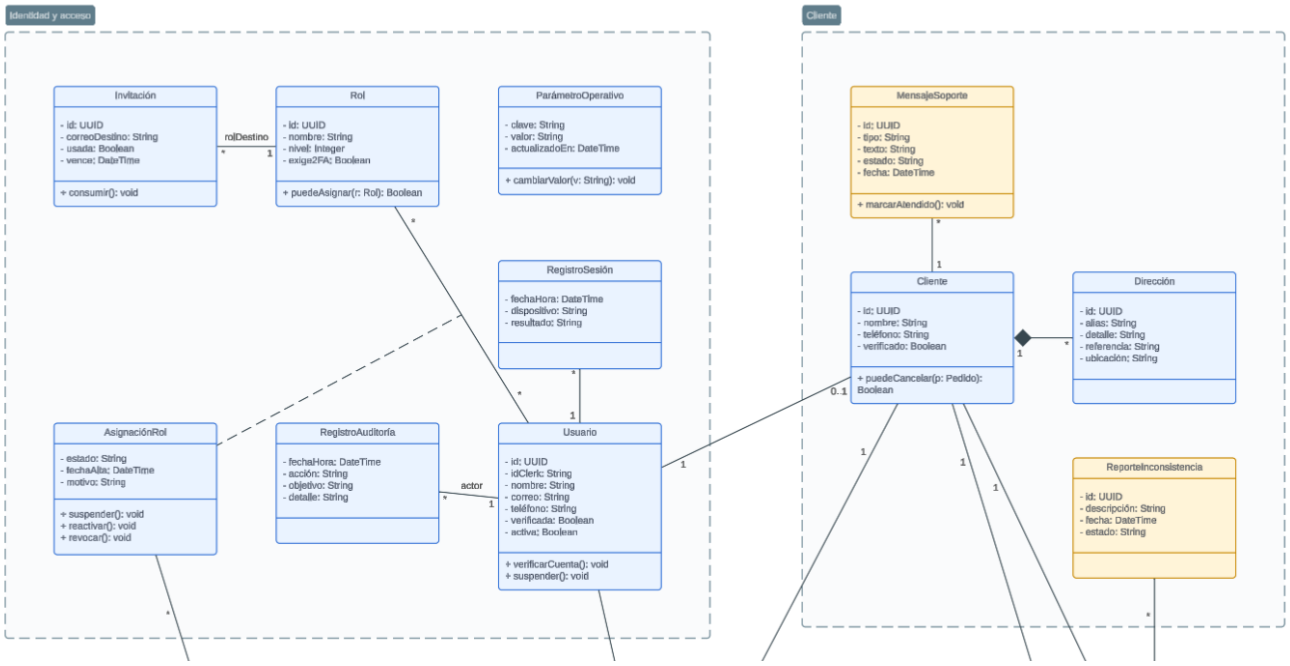
1.2.1. Diagrama de caso de usos del negocio

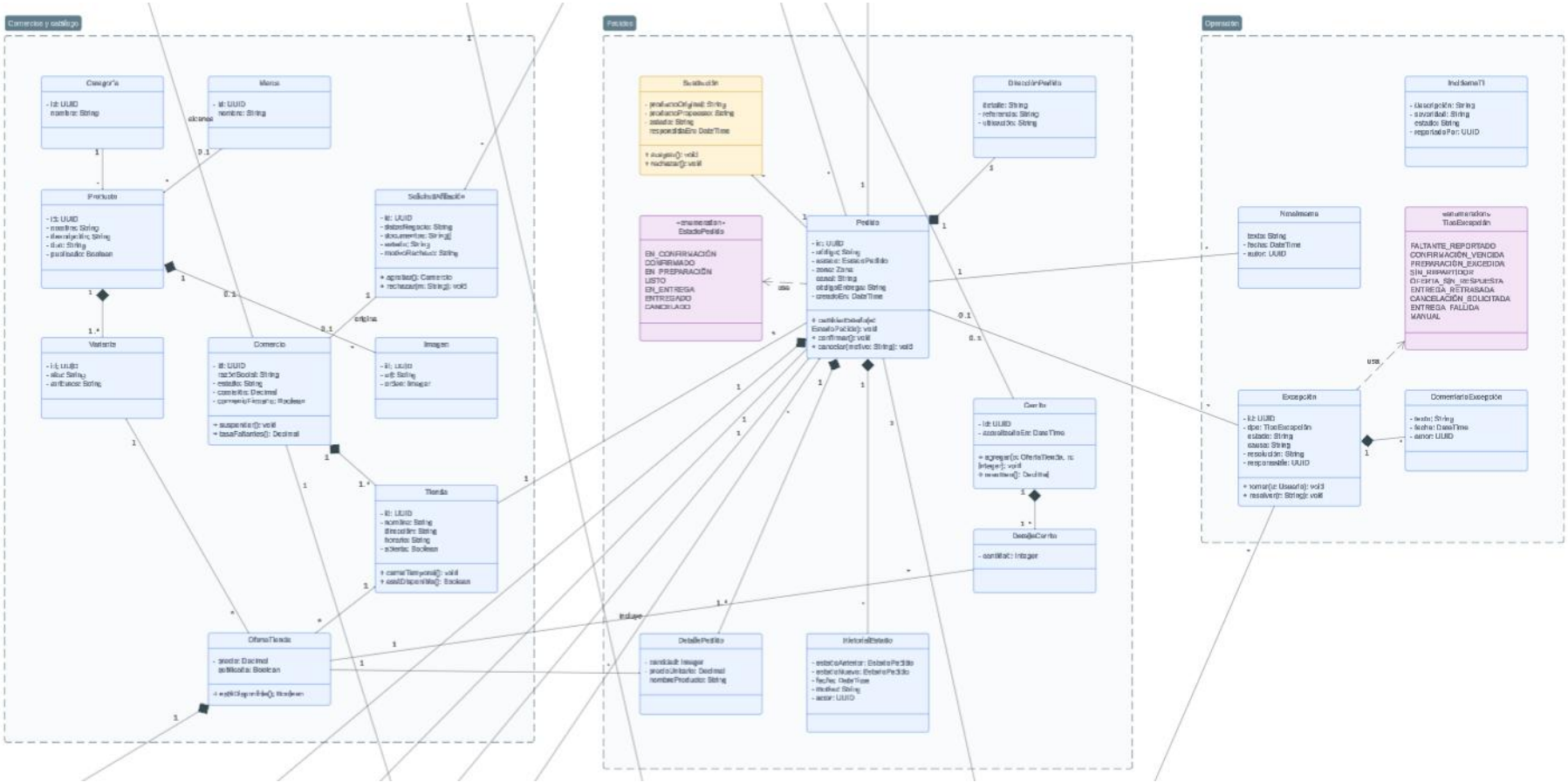


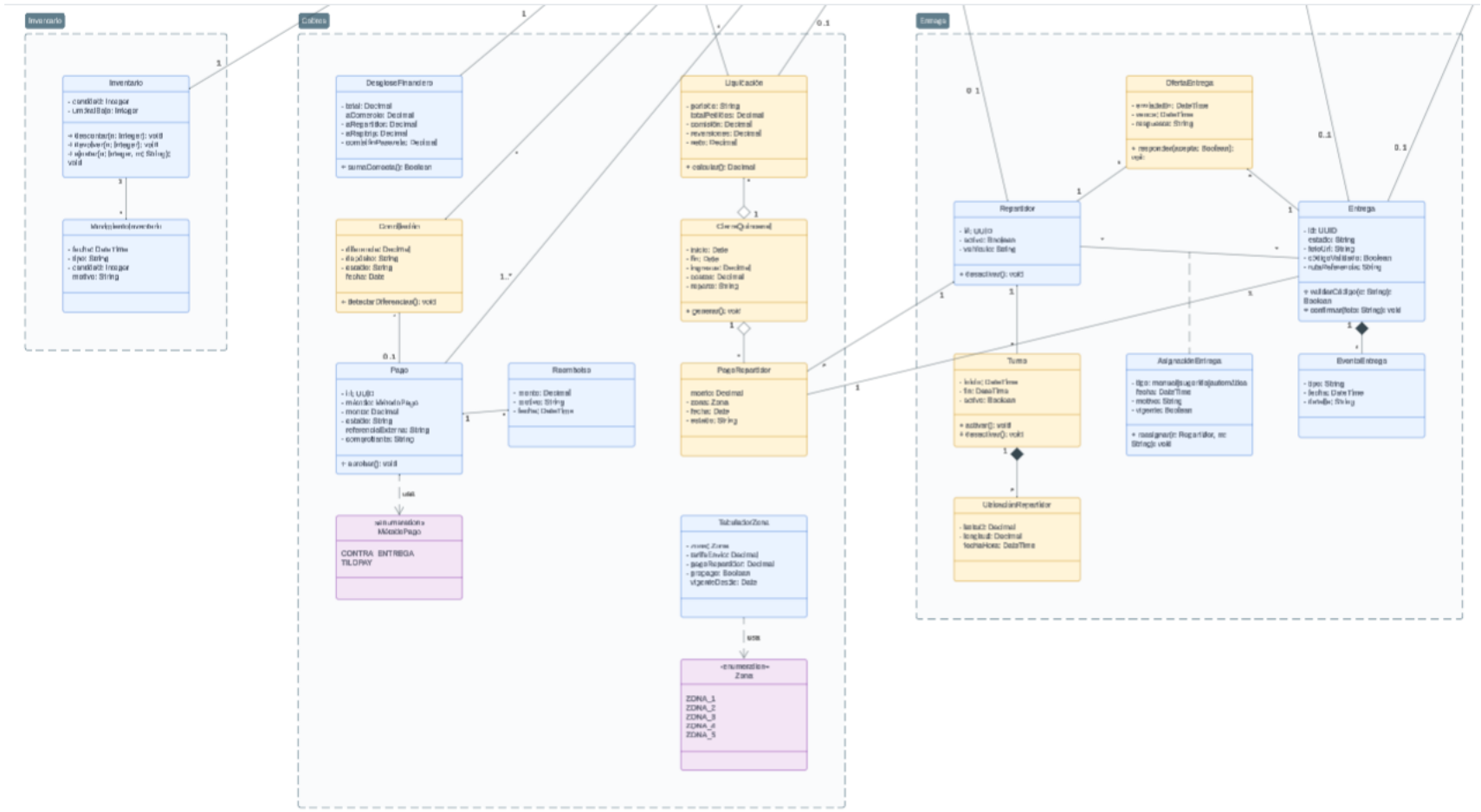
1.2.2. Diagrama de caso de usos del sistema



1.3. Diagrama de Clases




Comercios y catálogo
Pedidos
Operación
Categoría
Marca
Producto
SolicitudAfiliación
Variante
Comercio
Imagen
Tienda
OfertaTienda
SolicitudEdición
DirecciónPerfil
«enumeration»
EstadoPedido
Pedido
Carrito
DetalleCarrito
DetallePedido
HistorialEstado
IncidenciaTI
Novedades
«enumeration»
TipoExcepción
Excepción
ComentarioExcepción
usa
incluye


Inventario
Cobros
Entrega
Inventario
MovimientoInventario
DesgloseFinanciero
Liquidación
ContraEntrega
CierreQuincenal
Pago
Reembolso
PagoRepartidor
«enumeration»
MétodoPago
CONTRA_ENTREGA
TILOPAY
TabuladorZona
«enumeration»
Zona
ZONA_1
ZONA_2
ZONA_3
ZONA_4
ZONA_5
OfertaEntrega
Repartidor
Entrega
Turno
AsignacionEntrega
EventoEntrega
UbicacionRepartidor

1.4. Diagramas de navegación y presentación (Prototipo de figma)

El acápite para el diagrama de navegación y presentación se unificaron tomando en cuenta que para la lógica de ambos está basada en un prototipo de figma, de igual cumpliendo con lo que se requiere en la lógica de ambos diagramas presentando la navegación de los usuarios por el sistema y presentado tal cual una visualización inicial del sistema

1.4.1. Archivo de figma vista general de los protitipos y capas del sistema

Link de figma:

https://www.figma.com/design/ApaHyj069QzBBdk0EZvWF6/Rapitrip-%E2%80%94-Prototipo?node-id=154-16&t=t9E42RdjkWzE0mOy-1

1.4.2. Diagrama de navegación y presentación del cliente en Web

Link de figma para ver el prototipo:

https://www.figma.com/proto/ApaHyj069QzBBdk0EZvWF6/Rapitrip-%E2%80%94-Prototipo?node-id=154-1684&p=f&t=si0cF24jkYB08mGb-1&scaling=min-zoom&content-scaling=fixed&page-id=154%3A16&starting-point-node-id=154%3A1684&show-proto-sidebar=1

1.4.3. Diagrama de navegación y presentación de la consola de operación en Web

Link de figma para ver el prototipo:

https://www.figma.com/proto/ApaHyj069QzBBdk0EZvWF6/Rapitrip-%E2%80%94-Prototipo?node-id=123-27&p=f&t=s3xnbkQQvhMBgJSg-1&scaling=min-zoom&content-scaling=fixed&page-id=22%3A2&starting-point-node-id=123%3A27&show-proto-sidebar=1

1.4.4. Diagrama de navegación y presentación del repartidor en Movil

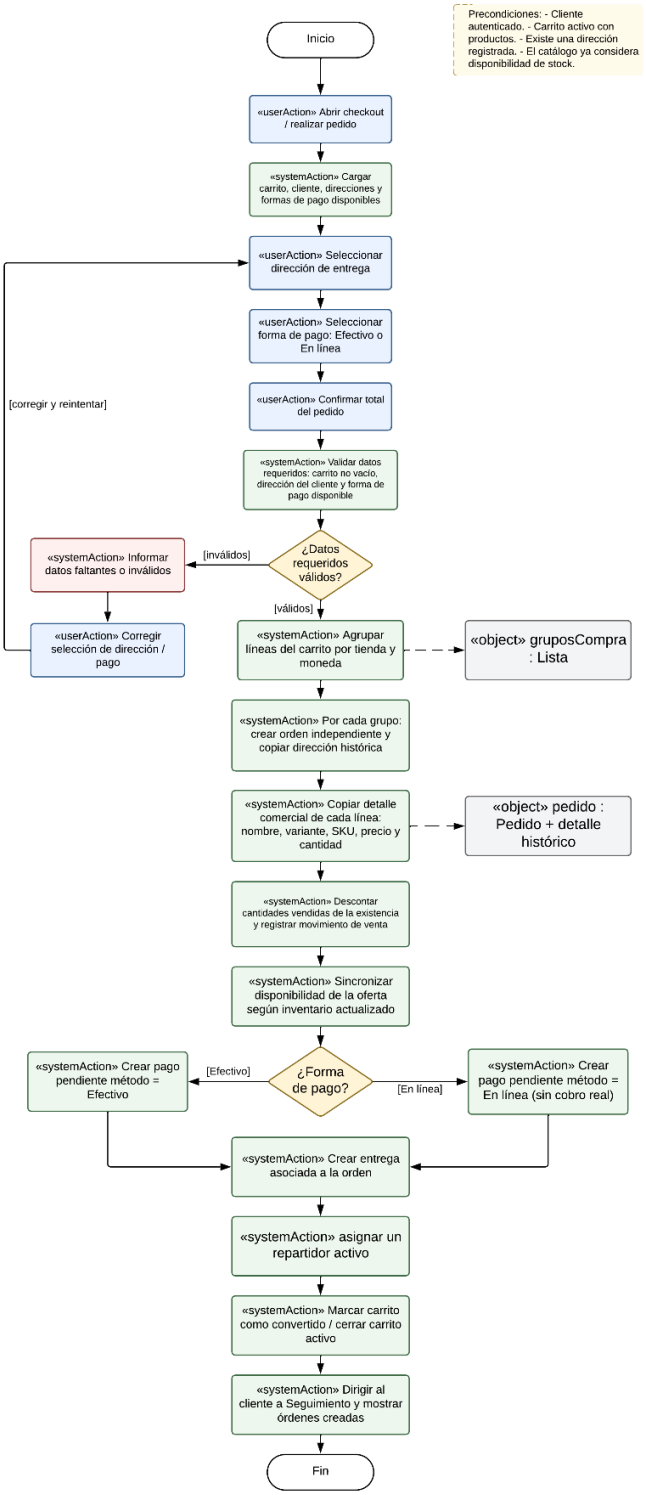
Link de figma para ver el prototipo:

https://www.figma.com/proto/ApaHyj069QzBBdk0EZvWF6/Rapitrip-%E2%80%94-

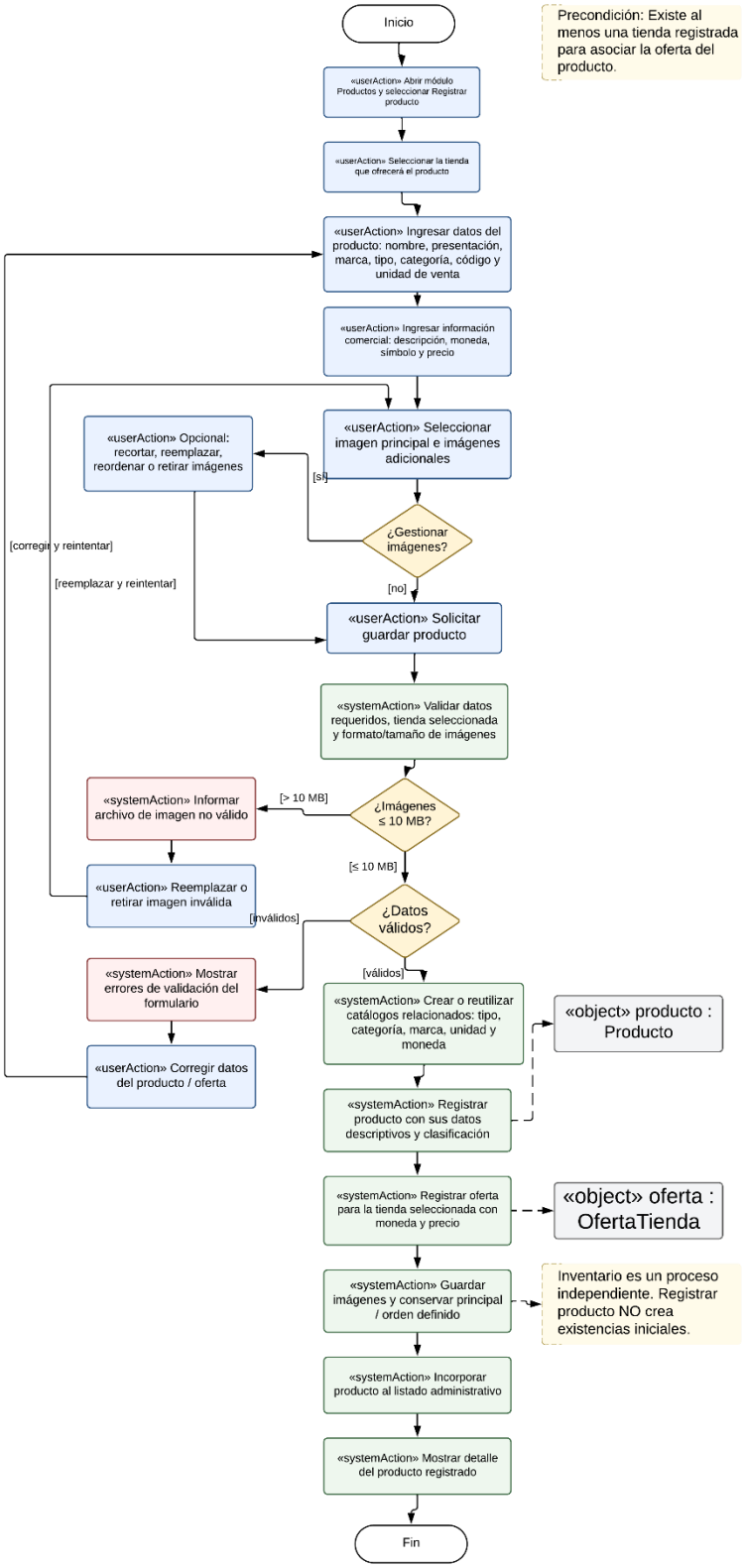
Prototipo?node-id=50-1140&p=f&t=NmKWPr5oVan7EQe3-1&scaling=min-zoom&content-scaling=fixed&page-id=50%3A1139&starting-point-node-id=50%3A1140&show-proto-sidebar=1

1.5. Diagramas de proceso

1.5.1. Diagrama de proceso | Realizar pedido



1.5.2. Diagrama de proceso | Registrar producto



1.6. Modelo Entidad – Relación

El modelo entidad-relación se organiza según las funciones principales de Rapitrip: identidad y acceso; comercios, catálogo e inventario; pedidos y entregas; parámetros operativos; pagos y caja; y, finalmente, auditoría y servicios internos. Cada recuadro representa una tabla, mientras que las líneas muestran las relaciones declaradas mediante claves foráneas en las migraciones. La clave primaria (PK) identifica cada registro y la clave foránea (FK) lo vincula con un registro de otra tabla.

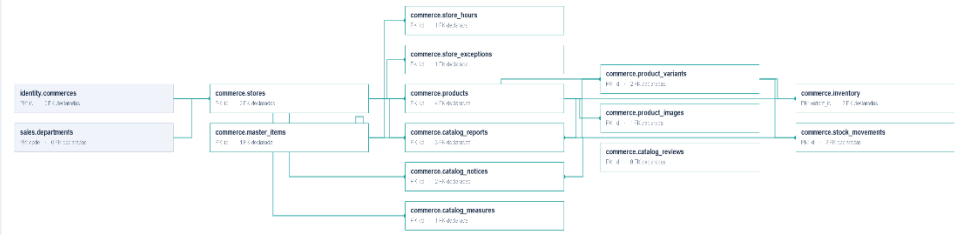
Esta estructura permite seguir el funcionamiento del sistema. Por ejemplo, un comercio tiene tiendas; las tiendas ofrecen productos y variantes; y esas variantes se relacionan con el inventario y los artículos de un pedido. A su vez, el pedido se vincula con el cliente, la entrega, el repartidor y los procesos de pago, devolución o facturación.

Se dividió el diagrama por áreas porque mostrar las 69 tablas y sus 98 claves foráneas en una sola red de líneas dificultaría la lectura. Algunas entidades aparecen en más de un área para facilitar el seguimiento de las conexiones; esto no significa que estén duplicadas en la base de datos. Además, las tablas sin líneas visibles no tienen una clave foránea declarada dentro de su área.

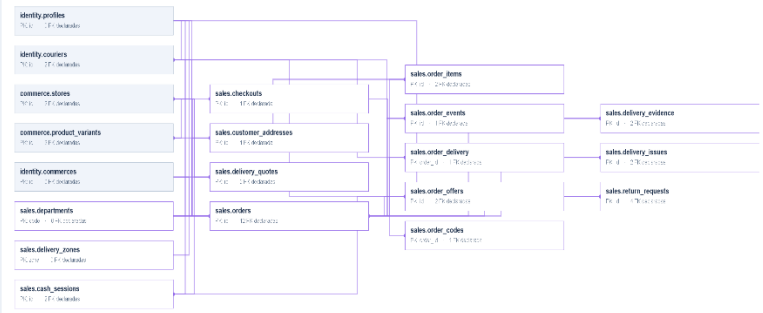
Identidad y acceso



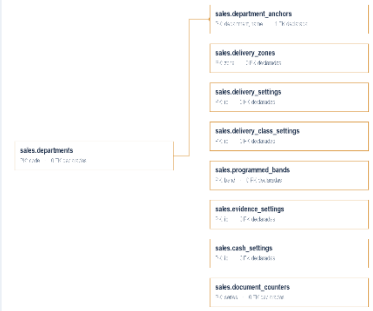
Comercios, catálogo e inventario



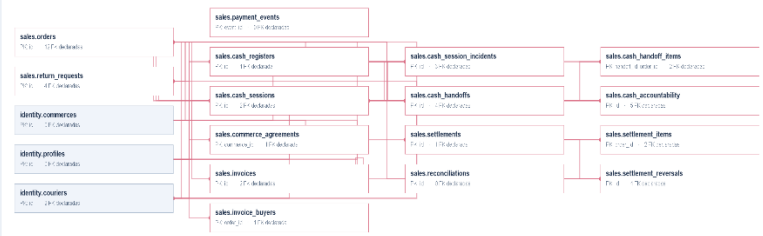
Pedidos y entregas



Geografía y parámetros operativos



Pagos, caja, liquidaciones y facturas



Auditoría y servicios internos



Capítulo II: Arquitectura de Software

2.1. Enfoque general

Rapitrip mantiene una arquitectura cliente-servidor: las personas utilizan aplicaciones web y las operaciones se resuelven mediante servicios del backend. Sin embargo, la implementación final no consiste en una única aplicación de servidor organizada como monolito. El sistema reúne tres aplicaciones web independientes en un mismo repositorio y un backend administrado en Supabase, cuya lógica se distribuye entre la base de datos, su API y funciones de borde.

Esta decisión conserva un núcleo de datos y reglas común para clientes, comercios, personal y repartidores. Al mismo tiempo, permite presentar interfaces distintas según las tareas de cada actor, sin duplicar las reglas críticas de acceso y de operación.

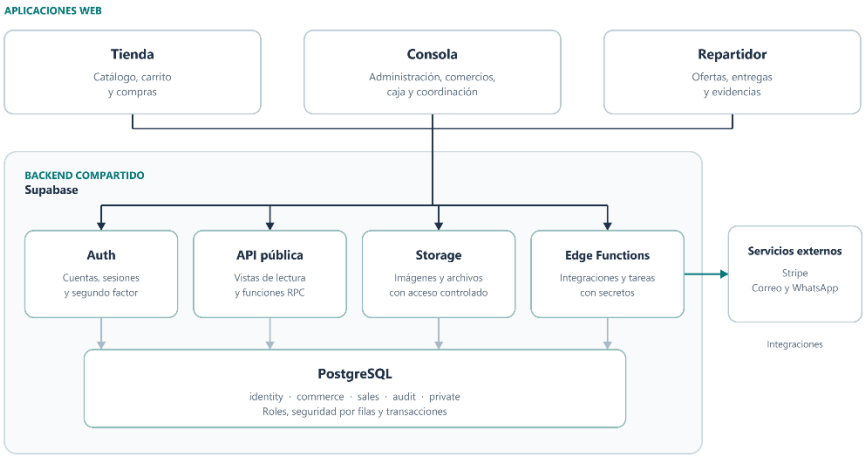


Figura 1. Estructura de aplicaciones y componentes del backend de Rapitrip.

2.2.Aplicaciones cliente y código compartido

La tienda web atiende al cliente y reúne el catálogo, el carrito, la compra y el seguimiento de pedidos. La consola web concentra las tareas de administración, caja, coordinación y gestión de comercios. Por su parte, la aplicación del repartidor permite consultar ofertas y entregas, registrar evidencias y gestionar acciones relacionadas con el efectivo. Las tres están desarrolladas con Next.js y se encuentran en las carpetas apps/store, apps/console y apps/dealer.

Estas aplicaciones comparten paquetes para componentes de interfaz, acceso a Supabase, validaciones, tipos de la base de datos y funciones de mapas. Por ello, la reutilización ocurre dentro del repositorio, mientras que cada aplicación conserva sus rutas, pantallas y responsabilidades propias. Los paquetes compartidos no constituyen servicios independientes desplegados por separado.

2.3. Backend y organización de los datos

Supabase proporciona la autenticación, la API, PostgreSQL, el almacenamiento de archivos y las Edge Functions. Las aplicaciones consumen servicios mediante el SDK de Supabase, tanto desde el navegador como desde código de servidor de Next.js. De esta forma, no establecen conexiones SQL directas con PostgreSQL: consultan vistas y ejecutan funciones públicas autorizadas, además de utilizar los servicios de autenticación y almacenamiento cuando corresponde.

La base de datos concentra las reglas que deben aplicarse de manera uniforme a todos los clientes. Sus tablas se agrupan por responsabilidad en los esquemas identity, commerce, sales, audit y private. El esquema public ofrece la superficie de acceso mediante vistas de lectura y funciones RPC; las políticas de seguridad por filas y las comprobaciones de permisos restringen las operaciones según la cuenta, el rol y su alcance.

En identity se gestionan perfiles, roles, comercios, invitaciones y repartidores. En commerce se organizan tiendas, horarios, productos, variantes e inventario. En sales se relacionan compras, pedidos, pagos, entregas, caja, liquidaciones y facturas. Además, audit registra eventos e incidentes, mientras que private contiene componentes internos como actividad de sesión y bandejas de salida. Esta separación funcional ordena el backend, aunque todos los módulos comparten la misma base de datos.

Las Edge Functions atienden operaciones que requieren secretos, comunicación externa o generación de archivos, como el procesamiento de eventos de pago, el envío de mensajes y la producción de documentos. Asimismo, Storage conserva imágenes y otros archivos con reglas de acceso acordes con su uso.

2.4. Recorrido de una operación

Una compra ilustra la relación entre las capas. El cliente prepara el carrito en la tienda y envía la solicitud de compra a una función pública. La base valida la información, calcula los importes y registra el checkout o el pedido junto con sus artículos, conforme al flujo elegido. Cuando interviene un pago con tarjeta, una Edge Function se comunica con Stripe y el webhook aplica el resultado mediante funciones de la base de datos. Después, la consola y la aplicación del repartidor consultan y actualizan únicamente las acciones que sus permisos permiten.

Así, el estado del pedido, el inventario, la entrega y los registros asociados se mantienen vinculados en PostgreSQL. Las notificaciones y otras tareas externas pueden utilizar bandejas de salida y ejecuciones programadas; este mecanismo asincrónico no implica que los módulos sean microservicios separados.

2.5.Seguridad y control de acceso

Supabase Auth gestiona las cuentas y sesiones. A partir de esa identidad, la base verifica los roles y aplica políticas de seguridad por filas para limitar qué información puede leerse o modificarse. Las acciones críticas se canalizan mediante funciones que comprueban permisos y condiciones de la operación. Para el personal y los comercios, el diseño también exige controles reforzados de sesión y segundo factor en la consola.

Los secretos de proveedores externos se reservan para las funciones y configuraciones de servidor. Por tanto, la interfaz utiliza únicamente las credenciales públicas y el contexto de la sesión; no debe decidir por sí sola si una persona puede ejecutar una acción sensible. La auditoría registra las operaciones relevantes para permitir su revisión posterior.

2.6. Ventajas y limitaciones del diseño

La principal ventaja es que los datos, permisos y transacciones se gestionan desde un núcleo común. Esto favorece la consistencia de pedidos, inventario y pagos, a la vez que permite reutilizar componentes y validaciones entre las aplicaciones. Además, la división por esquemas y funciones hace explícitas las responsabilidades sin exigir la operación de múltiples bases de datos o servicios propios.

Como limitación, parte de la lógica queda ligada a PostgreSQL y a los servicios de Supabase, por lo que cualquier cambio en las funciones o en el modelo de datos puede afectar a varios clientes. También es necesario coordinar la evolución de las aplicaciones y las migraciones, comprobar la compatibilidad de sus contratos y supervisar las integraciones externas. En consecuencia, la independencia de despliegue de las interfaces no elimina la dependencia del backend compartido.

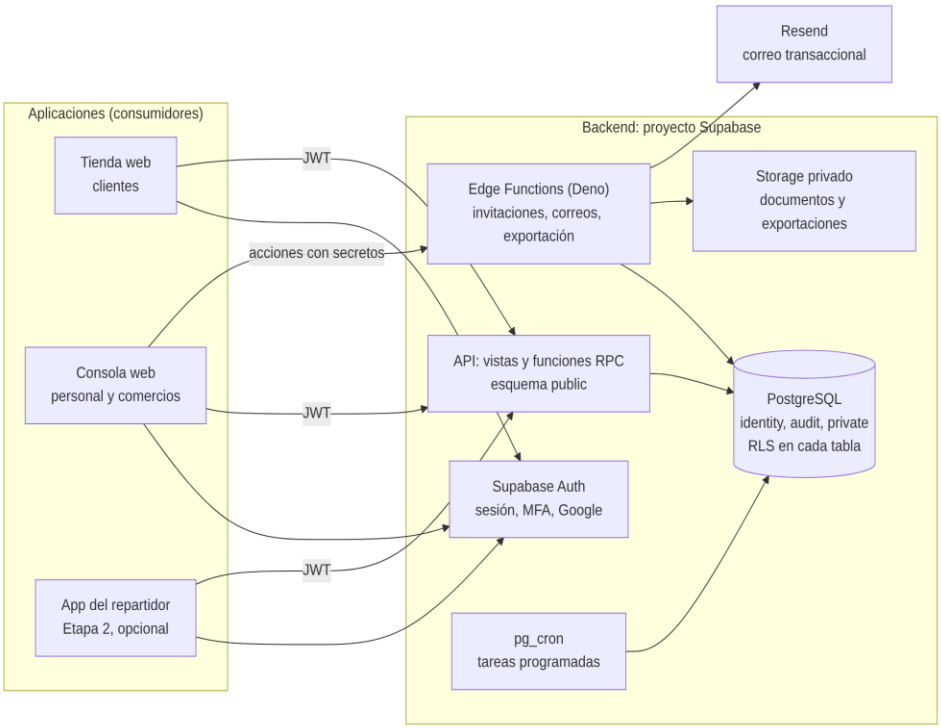
2.7. Evolución prevista

La arquitectura actual puede ampliarse manteniendo límites claros entre los módulos de identidad, comercio, ventas y logística. Antes de separar alguno como servicio independiente, conviene medir problemas reales de escala, disponibilidad o ritmo de cambios. Si aparecen esas necesidades, se podrán evaluar mecanismos adicionales de integración y despliegue. Por ahora, los webhooks, las bandejas de salida y las tareas programadas resuelven procesos asincrónicos concretos sin convertir al sistema en una arquitectura de microservicios.

Capítulo III: Desarrollo de backend

Rapitrip es una plataforma con tres aplicaciones web: una tienda para los clientes, una consola para el personal y los comercios, y una aplicación para los repartidores. Las tres comparten un backend administrado en Supabase, donde se gestionan los datos, los permisos y las reglas críticas de operación. Las interfaces consumen sus servicios y conservan la lógica de presentación y validación necesaria para cada actor.

Figura 1.
Vista general del backend y sus consumidores



3.1. Tecnología utilizada

3.1.1. Tecnologías por capa

Tabla 1.
Tecnologías del backend

Capa	Tecnología	Uso
Plataforma	Supabase	Base de datos, autenticación, funciones del servidor, almacenamiento y tareas programadas
Base de datos	PostgreSQL	Datos del dominio, transacciones y reglas de seguridad
Seguridad de datos	Row Level Security (RLS)	Cada consulta ve solo las filas que permiten su rol y su alcance

Capa	Tecnología	Uso
Lógica de negocio	Funciones SQL (RPC) en PL/pgSQL	Todas las escrituras y las reglas críticas, dentro de la base de datos
Lógica con secretos	Edge Functions en Deno y TypeScript	Llamadas a servicios externos y uso de claves del servidor
Identidad	Supabase Auth	Registro, inicio de sesión, tokens JWT, segundo factor (TOTP) y acceso con Google
Tareas programadas	pg_cron, pg_net y Vault	Sincronizar inicios de sesión, detectar incidentes, marcar cuentas inactivas y enviar correos
Archivos	Supabase Storage	Documentos de repartidores y archivos de exportación, en depósitos privados
Correo	Resend (SMTP y API)	Confirmación de correo, invitaciones, recuperación de contraseña y avisos
Protección contra abuso	Cloudflare Turnstile (CAPTCHA)	Frena la fuerza bruta en el inicio de sesión de la consola
Pruebas	pgTAP, Deno test, Vitest	Base de datos, funciones del servidor y paquetes compartidos
Lenguajes	SQL / PL/pgSQL y TypeScript	SQL para las reglas; TypeScript para funciones, validaciones y scripts
Validación	zod	Repite en las interfaces las reglas de la base para avisar antes de llamar al servidor
Repositorio	Monorepo con pnpm, Git y GitHub	Backend y paquetes comunes en un solo repositorio versionado
Herramientas	Supabase CLI y Docker	Entorno local igual al de producción y aplicación de migraciones
Integración continua	GitHub Actions	Corre todas las pruebas en cada cambio

3.1.2. Supabase como backend central

El backend se construye sobre Supabase y no sobre un servidor propio con Node, Express o NestJS. Las razones:

- Hay menos que operar. Para un equipo de tres personas y 25 a 30 pedidos al mes, un servicio administrado evita mantener servidores, balanceadores y parches.

- La seguridad vive junto a los datos. Con RLS y funciones en la base, las reglas se aplican igual sin importar qué aplicación llame, y un canal nuevo (la app del repartidor o una app móvil del cliente) no obliga a reescribir permisos ni estados.
- Las transacciones son atómicas. Una acción y su registro de auditoría se confirman o se revierten juntos, algo que un servidor aparte tendría que coordinar a mano.
- Se usa un solo lenguaje de aplicación: TypeScript en funciones, validaciones y scripts, y SQL para las reglas.

El costo es que la lógica queda en PL/pgSQL y en Edge Functions, y se prueba con pgTAP y no con las herramientas habituales de un servidor. El diseño lo compensa exigiendo una prueba por cada regla de acceso.

3.2. Patrón de diseño que se utilizó

3.2.1. Backend modular centrado en la base de datos

El backend se organiza por dominios sobre un proyecto compartido de Supabase. PostgreSQL concentra los datos y las reglas críticas, mientras que las Edge Functions atienden integraciones y tareas que requieren secretos. No se emplean microservicios propios; las tres aplicaciones web se construyen y despliegan por separado.

Cada módulo es un esquema de PostgreSQL con sus tablas, políticas y funciones.

Tabla 2.
Esquemas del backend

Esquema	Responsabilidad	Acceso desde fuera
identity	Perfiles, roles, invitaciones, solicitudes de comercio y fichas de repartidores	Solo por vistas y RPC
audit	Auditoría, inicios de sesión e incidentes de TI	Solo por RPC; sin escritura directa
private	Funciones de seguridad y auxiliares, cola de correos	Ninguno
public	Vistas de lectura y funciones RPC	Aplicaciones, con permisos explícitos

Los dominios de identidad, comercios, catálogo, inventario y pedidos se incorporaron mediante migraciones, manteniendo límites funcionales dentro de la base de datos compartida.

3.2.2. Patrones aplicados

Tabla 3.
Patrones de diseño y su aplicación

Patrón	**Aplicación en el backend**
Control de acceso por roles con alcance (RBAC)	Seis roles. El rol comercio queda limitado a su comercio. La matriz de quién gestiona a quién es una tabla de datos, private.role_management, y no código repetido
Defensa en profundidad y Zero Trust	Ninguna aplicación es confiable. Cada acción se valida en la base: rol activo, alcance, sesión de consola y, cuando corresponde, segundo factor reciente
Denegado por defecto	Toda tabla nace con RLS activo y sin políticas, y ninguna función nueva es ejecutable hasta que se concede
API por funciones (fachada)	Las aplicaciones no escriben en tablas. Llaman funciones RPC de nombre estable que validan y ejecutan
Separación de lectura y escritura (CQRS ligero)	Las lecturas pasan por vistas con security_invoker y RLS; las escrituras, solo por RPC
Funciones con privilegios elevados encapsuladas	Las funciones SECURITY DEFINER viven en un esquema no expuesto, fijan search_path y toman la identidad de auth.uid() en lugar de recibirla como parámetro
Registro de auditoría de solo inserción (event log)	Una función única escribe la auditoría dentro de la transacción de la acción. Un trigger impide modificar o borrar, incluso al propietario
Minimización de datos	La auditoría guarda ids y nombres de campos, nunca valores personales. Los datos personales viven en una tabla aparte, profile_pii
Transactional Outbox	Los correos que nacen en la base se encolan en la misma transacción y una función los envía después, con reintentos y sin duplicados
Token de un solo uso con hash	Las invitaciones guardan solo el hash SHA-256 del token, que se consume una vez
Pseudonimización	Al anonimizar una cuenta se borran los datos personales y el acceso, y el perfil queda como identificador anónimo para pedidos y cobros
Contrato por tipos generados	Los tipos de TypeScript se generan desde el esquema real y son el contrato con las aplicaciones. La integración continua falla si quedan desactualizados

3.3. Estrategias de desarrollo

3.3.1. Metodología

El desarrollo sigue Scrum adaptado a tres personas, como define la propuesta tecnológica: sprints de dos semanas con 180 horas de capacidad (3 integrantes por 30 horas durante dos semanas) y una revisión con un administrador de Rapitrip al cierre de cada sprint. Una tarea está terminada cuando otro integrante revisó el código, las pruebas automáticas pasan y los permisos se verificaron con la matriz de pruebas.

3.3.2. Secuencia de construcción

El backend va un paso adelante de la interfaz. Se trabaja por rebanadas verticales:

1. Se construye primero el backend completo de la fase F1 (identidad, roles y auditoría) con sus pruebas, porque todo lo demás depende de él.
2. Después se avanza módulo por módulo, con el backend de cada uno un sprint antes que su pantalla.
3. El contrato entre ambos son los tipos generados y las funciones RPC de nombre estable. Una pantalla nunca depende de una tabla que aún no existe.

Con tres personas, una puede preparar el backend del módulo siguiente mientras las otras dos hacen la interfaz del anterior.

3.3.3. Desarrollo guiado por pruebas

Cada tarea sigue el mismo ciclo: escribir la prueba, verla fallar, implementar lo mínimo, verla pasar y registrar el cambio.

Tabla 5.
Cobertura de pruebas

Qué se prueba	**Herramienta**
Matriz de permisos: cada tabla, vista y función por cada rol, incluido el anónimo	pgTAP
Matriz de gestión de roles, en positivo y en negativo	pgTAP

Qué se prueba	Herramienta
Restricciones de la base: un comercio por cuenta y rol superadmin solo por código	pgTAP
Auditoría inmutable: modificar o borrar falla para todos los roles	pgTAP
Sesión de consola con tokens simulados y con tokens reales de Auth	pgTAP y scripts de verificación
Funciones del servidor: tokens, plantillas y CSV	Deno test
Validaciones compartidas	Vitest

Una prueba de regresión hace fallar el cambio si aparece una tabla sin RLS, una función ejecutable por el rol anónimo o una función con privilegios elevados sin search_path fijo.

3.3.4. Seguridad desde el diseño

- Mínimo privilegio: cada función se concede una por una, y las de servidor solo las ejecuta el rol interno.
- Datos sensibles: el documento de identidad del repartidor, los datos personales completos de un cliente y la consulta de la auditoría ajena se leen solo por funciones que registran la lectura.
- Exportaciones: las hacen solo el administrador y el superadministrador, con motivo, segundo factor de los últimos 5 minutos, archivo cifrado y enlace de 15 minutos.
- Sesión: segundo factor obligatorio en todos los roles de consola y cierre por inactividad en dos capas, la de Supabase y una propia que registra la actividad real.
- Ley 787: anonimización de cuentas inactivas (12 meses, con confirmación del administrador), atención de solicitudes de los titulares y auditoría sin datos personales.

3.3.5. Control de cambios y entornos

- Todo cambio de esquema es una migración SQL con nombre fijo. Es la única fuente de verdad y solo avanza.

- Hay dos entornos: local, igual al de producción, con la CLI de Supabase y Docker, y un proyecto de producción. El entorno de staging queda para una planificación posterior, cuando haya clientes reales.
- En cada cambio la integración continua levanta Supabase, aplica las migraciones, corre las pruebas, revisa la base con el lint de Supabase y verifica que los tipos generados estén al día.
- Se hace un commit por tarea completada, con su prueba incluida.

3.3.6. Fases del proyecto

Este apartado se originó en la fase F1 y conserva la secuencia de construcción prevista en el plan inicial. Los módulos desarrollados posteriormente se describen en el análisis y la arquitectura integrados de este entregable.

Tabla 6.
Fases y backend

Fase	**Alcance**	Enfoque del plan inicial
F1	Identidad, roles, permisos, auditoría y consola administrativa	Base del backend
F2	Catálogo por tienda, inventario y horarios	Módulo siguiente
F3	Motor de pedidos, cobros con Stripe, caja y venta	Después de F2
F4	Tienda web del cliente y puesta en marcha de la Etapa 1	Interfaz
F5 a F7	Aplicación web del repartidor, tablero de operaciones y liquidaciones	Etapa 2

5. URL de proyecto en github

Enlace: https://github.com/richard0956/Proyecto-Rapitrip.git